

LEVEL 9 PROJECT DOCUMENTATION

KUBERNETES CLUSTER DEPLOYMENT AND FLASK CALCULATOR APPLICATION

AWS EC2 + kOps + Kubernetes + Cilium + Docker Hub + S3 + NodePort

Environment: Amazon Linux 2023

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Cluster: level9.k8s.local

Prepared by Manimaran

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives
- 4. Technologies and Services
- 5. Prerequisites
- 6. System Architecture
- 7. Complete Kubernetes Workflow
- 8. Project Components and Configuration
- 9. Complete Implementation Procedure – 34 Screenshot Actions
- 10. Kubernetes Deployment and Service Design
- 11. Rolling Update and Rollback
- 12. Application Request Flow
- 13. Validation and Verification Checklist
- 14. Troubleshooting and Operational Notes
- 15. Security Notes
- 16. Key Lessons Learned
- 17. Interview / Viva Explanation
- 18. Conclusion
- Appendix – Complete Project Flow

1. Project Overview

This Level 9 project demonstrates deployment of a Kubernetes cluster on AWS using kOps and Amazon EC2. Cluster networking is provided by Cilium. A Flask calculator application is deployed as a Kubernetes Deployment with three replicas. A NodePort service exposes the application on port 30080 and forwards traffic to the application container on port 5000. The project also covers Docker Hub image pulling with an imagePullSecret, Amazon S3 as the kOps state store, rolling updates, rollback testing and browser verification.

2. Problem Statement

Creating and managing a Kubernetes cluster manually requires configuring AWS compute resources, cluster state storage, Kubernetes components, networking, application workloads and external access. This project uses kOps to automate the AWS Kubernetes cluster lifecycle and Kubernetes resources to manage the application deployment, service exposure, rolling updates and rollback.

3. Objectives

Create and validate a Kubernetes cluster on AWS using kOps; use Amazon S3 as the kOps cluster state store; provision Kubernetes infrastructure on EC2; use Cilium as the Kubernetes networking/CNI layer; deploy the Flask calculator application; run three application replicas; authenticate to Docker Hub with an imagePullSecret; expose the application through NodePort 30080 → container port 5000; verify browser access; and test rolling update and rollback behavior.

4. Technologies and Services

Technology / Service	Purpose / Role
AWS EC2	Hosts the kOps cluster infrastructure and administration environment
Amazon Linux 2023	Operating-system environment shown in the project
kOps 1.36.2	Creates, manages and validates the Kubernetes cluster on AWS
Kubernetes	Container orchestration platform
kubectl v1.37.0	Kubernetes command-line administration client
Cilium	Kubernetes pod networking / CNI
Amazon S3	kOps cluster state store; bucket 1kops
Docker Hub	Container image registry
imagePullSecret	Kubernetes registry authentication for Docker Hub
Flask	Calculator web application
NodePort	External Kubernetes Service exposure on port 30080

5. Prerequisites

- AWS account with EC2, S3 and required Kubernetes/kOps permissions.
- Amazon Linux 2023 EC2 administration environment.
- AWS CLI installed and configured for ap-south-1.
- kOps and kubectl installed and version verified.
- SSH key pair available for kOps-managed nodes.
- Docker Hub account/registry credentials for imagePullSecret.
- Flask calculator container image available in Docker Hub.
- Security-group access to TCP 30080 for browser testing.

The diagram illustrates the architecture of a VPC (kOps-managed) and its interaction with external services. The VPC contains a Control Plane (c7i-flex.large) and a Worker Node (c7i-flex.large). The Control Plane includes kube-apiserver, controller-mgr, scheduler, and etcd. The Worker Node includes kubelet, containerd, Cilium (CNI), and EBS CSI driver. The Control Plane and Worker Node are connected via etcd/API. The Control Plane is connected to an ELB (API LB) via kube-apiserver (via ELB, TLS/443). The Worker Node is connected to the ELB via NodePort:30080. The Worker Node runs Calculator App Pods (x3) using Flask on port 5000. The End User (Browser/curl) interacts with the Calculator App Pods via Cilium (CNI) handles pod networking. The End User also interacts with the S3 Cluster State Store via kubectl / kOps CLI. The S3 Cluster State Store is connected to the Worker Node. The Worker Node is connected to Docker Hub (private repo) via image pull via imagePullSecret.

7. Complete Kubernetes Workflow

Step	Flow	Purpose
1	AWS/EC2 administration	Prepare the Amazon Linux environment and AWS CLI.
2	S3 state store	Use bucket 1kops as KOPS_STATE_STORE.
3	kOps cluster configuration	Create level9.k8s.local cluster configuration.
4	Cluster creation	Run kops update cluster --yes and wait for AWS/Kubernetes resources.
5	Cluster validation	Use kops validate cluster, kubectl get nodes and kubectl get pods -A.
6	Cilium/system components	Verify Cilium, CoreDNS, etcd, API server, controller/scheduler and EBS CSI workloads.
7	Docker Hub authentication	Create dockerhub-secret imagePullSecret.
8	Application Deployment	Create calculator-app with three replicas.
9	NodePort Service	Expose calculator-service on NodePort 30080 → port 5000.
10	Browser validation	Test the calculator through the worker public IP and port 30080.
11	Rolling update	Apply a new image/version and observe ReplicaSet transition.
12	Rollback	Use kubectl rollout undo and verify the previous version.
13	Cleanup	Delete the kOps cluster after testing.

8. Project Components and Configuration

Parameter	Project Value
Cluster name	level9.k8s.local
kOps state store	S3 bucket 1kops
kOps version	1.36.2
kubectl client	v1.37.0
Networking	Cilium
Deployment	calculator-app
Replicas	3
Service	calculator-service
Service type	NodePort
NodePort	30080
Application/container port	5000
Registry	Docker Hub
Image pull secret	dockerhub-secret

9. Complete Implementation Procedure – 34 Screenshot Actions

Following the Level 7 format, each screenshot is preceded by an Implementation Action / What I did statement and a What the screenshot proves statement. The evidence remains in chronological project order. Credential-bearing screenshots are embedded in redacted form for security.

Step 1 : Verify the kOps S3 state-store bucket

Implementation Action / What I did: Open Amazon S3 in ap-south-1 and confirm the bucket named 1kops is present. This bucket is used as the kOps state store.

What the screenshot proves: The 1kops bucket is visible in the S3 bucket list.

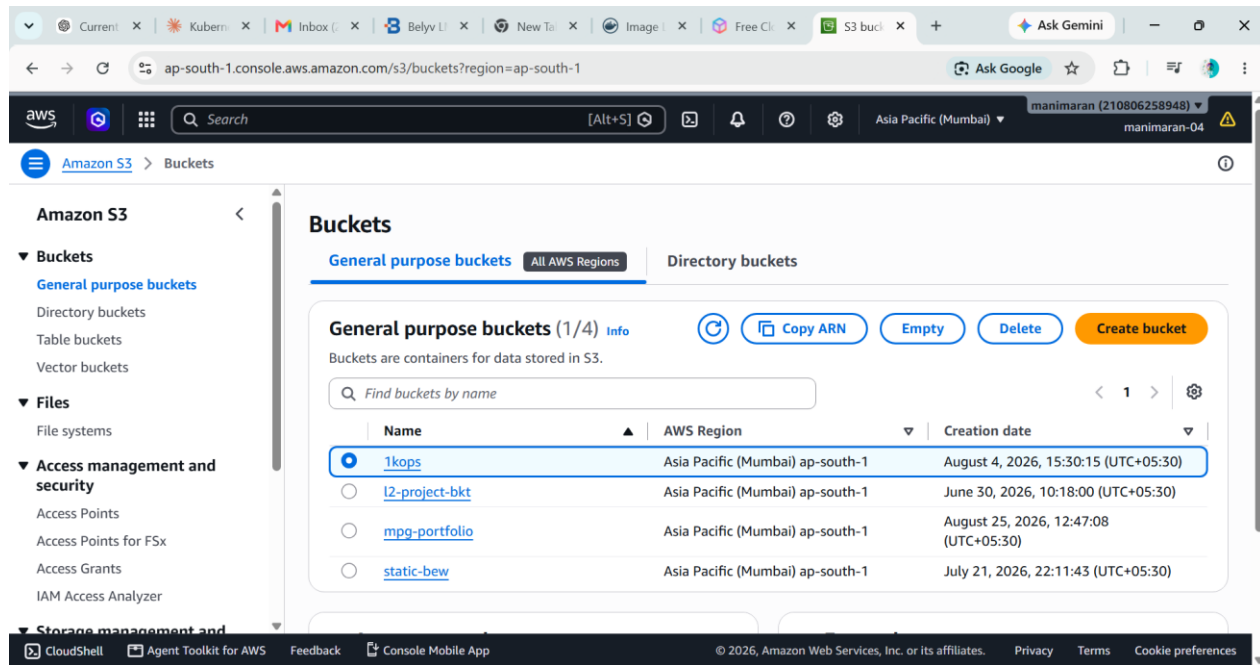


Figure 2. Verify the kOps S3 state-store bucket.

Step 2 : Launch the EC2 environment for kOps administration

Implementation Action / What I did: Open EC2 Launch Instance, select the Amazon Linux 2023 AMI and the c7i-flex.large instance type shown in the evidence. Configure the required network/security settings and launch the instance.

What the screenshot proves: The EC2 launch configuration is prepared with Amazon Linux 2023 and c7i-flex.large.

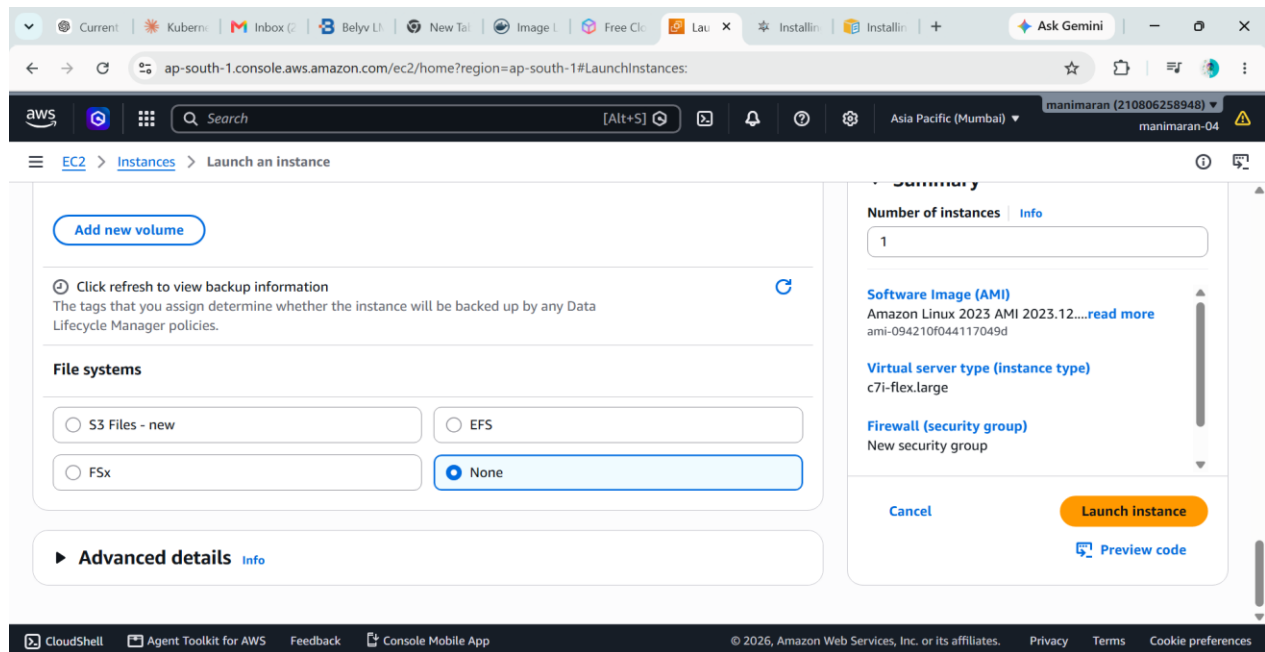


Figure 3. Launch the EC2 environment for kOps administration.

Step 3 : Verify the kOps administration instance

Implementation Action / What I did: Open the EC2 Instances page and confirm the kOps administration instance is running and passing its status checks.

What the screenshot proves: The kOps EC2 instance is running in ap-south-1.

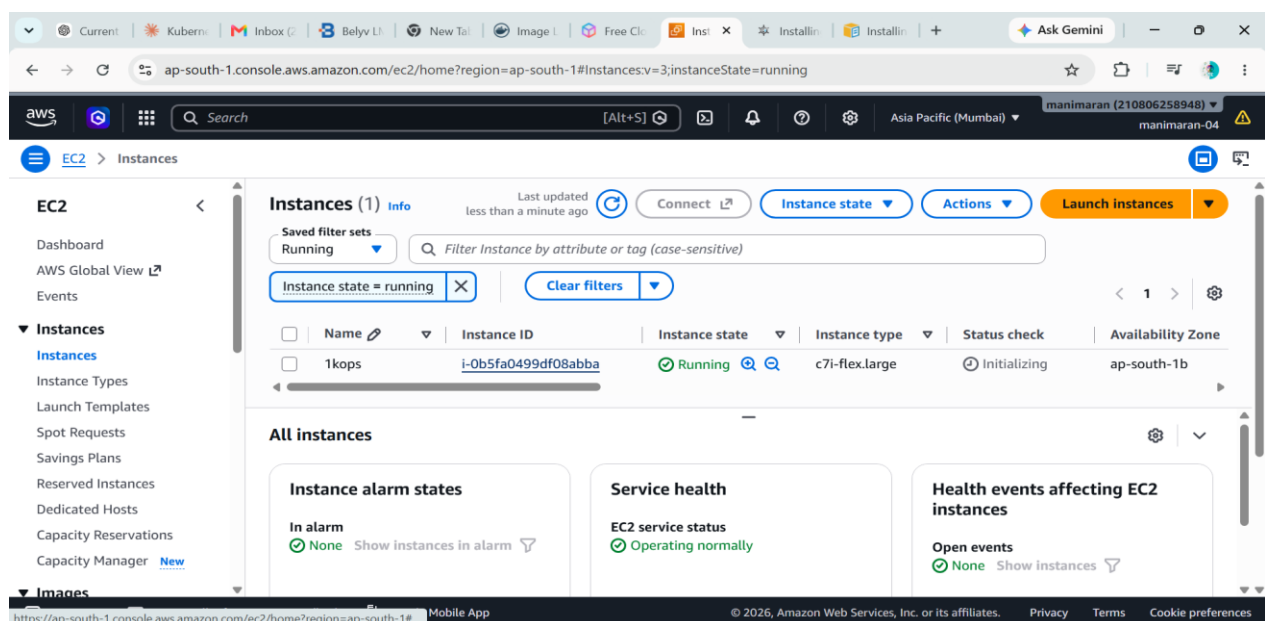
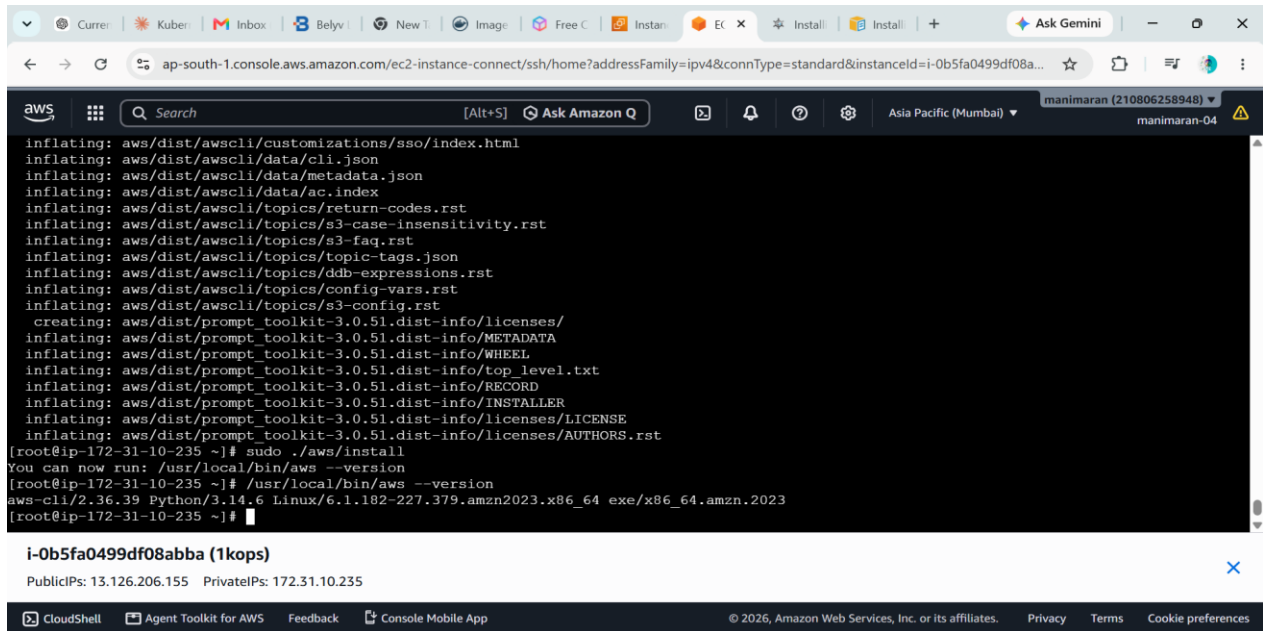


Figure 4. Verify the kOps administration instance.

Step 4 : Install AWS CLI v2

Implementation Action / What I did: From the EC2 terminal, install AWS CLI v2 and verify the installation with `/usr/local/bin/aws --version`.

What the screenshot proves: AWS CLI v2 is installed and its version is displayed.



```

inflating: aws/dist/awscli/customizations/sso/index.html
inflating: aws/dist/awscli/data/cli.json
inflating: aws/dist/awscli/data/metadata.json
inflating: aws/dist/awscli/data/ac.index
inflating: aws/dist/awscli/topics/return-codes.rst
inflating: aws/dist/awscli/topics/s3-case-insensitivity.rst
inflating: aws/dist/awscli/topics/s3-faq.rst
inflating: aws/dist/awscli/topics/topic-tags.json
inflating: aws/dist/awscli/topics/ddb-expressions.rst
inflating: aws/dist/awscli/topics/config-vars.rst
inflating: aws/dist/awscli/topics/s3-config.rst
creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
[root@ip-172-31-10-235 ~]# sudo ./aws/install
You can now run: /usr/local/bin/aws --version
[root@ip-172-31-10-235 ~]# /usr/local/bin/aws --version
aws-cli/2.36.39 Python/3.14.6 Linux/6.1.182-227.379.amzn2023.x86_64 exe/x86_64.amzn.2023
[root@ip-172-31-10-235 ~]#

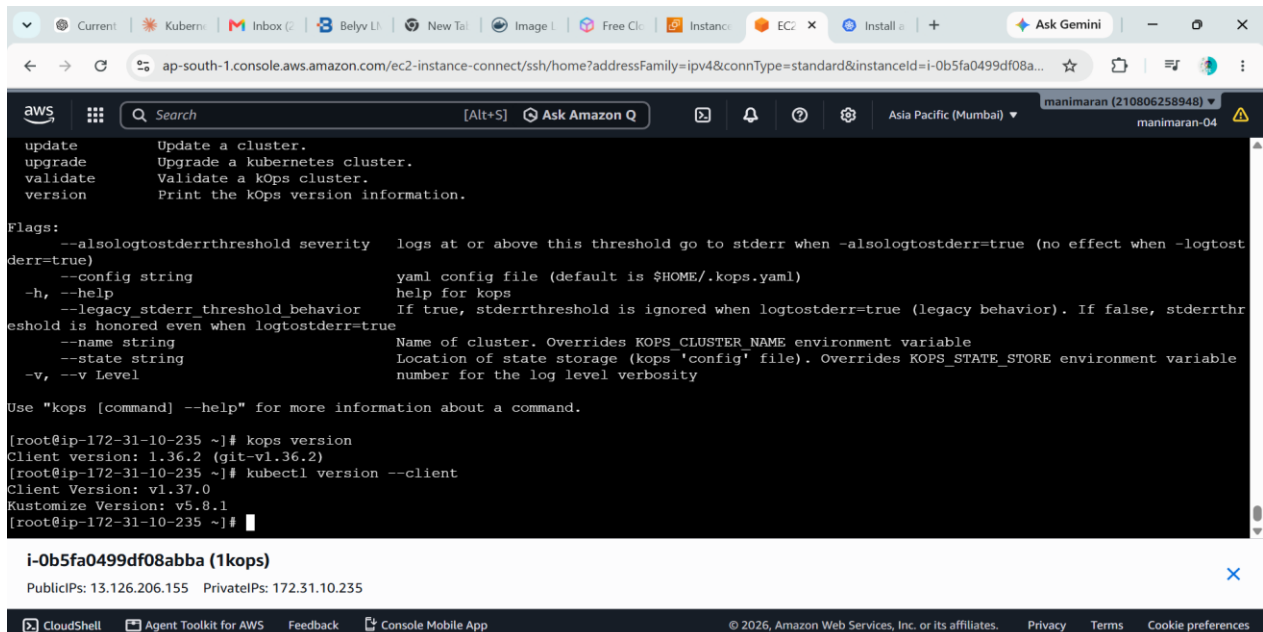
```

Figure 5. Install AWS CLI v2.

Step 5 : Install and verify kOps and kubectl

Implementation Action / What I did: Run `kops version` and `kubectl version --client` to verify the tools required for cluster creation and administration. The supplied evidence shows kOps 1.36.2 and kubectl v1.37.0.

What the screenshot proves: The required kOps and kubectl client versions are available.



```

update      Update a cluster.
upgrade     Upgrade a kubernetes cluster.
validate    Validate a kOps cluster.
version     Print the kOps version information.

Flags:
  --alsologtostderr threshold severity  logs at or above this threshold go to stderr when -alsologtostderr=true (no effect when -logtostderr=true)
  --config string                       yaml config file (default is $HOME/.kops.yaml)
  -h, --help                             help for kops
  --legacy_stderr_threshold_behavior    If true, stderrthreshold is ignored when logtostderr=true (legacy behavior). If false, stderrthreshold is honored even when logtostderr=true
  --name string                         Name of cluster. Overrides KOPS_CLUSTER_NAME environment variable
  --state string                       Location of state storage (kops 'config' file). Overrides KOPS_STATE_STORE environment variable
  -v, --v Level                         number for the log level verbosity

Use "kops [command] --help" for more information about a command.

[root@ip-172-31-10-235 ~]# kops version
Client version: 1.36.2 (git-v1.36.2)
[root@ip-172-31-10-235 ~]# kubectl version --client
Client Version: v1.37.0
Kustomize Version: v5.8.1
[root@ip-172-31-10-235 ~]#

```

Figure 6. Install and verify kOps and kubectl.

What the screenshot proves: AWS CLI is configured for the project region ap-south-1. The generated documentation redacts credential values from this evidence.



Implementation Action / What I did: Run `aws s3 ls` and confirm that the 1kops bucket is listed.

Figure 8. Verify the S3 state store from the CLI.

Step 8 : Prepare SSH keys for kOps

Implementation Action / What I did: Create the ~/.ssh directory/key files required by kOps and ensure the SSH public key is available for cluster nodes. Do not expose the private key contents.

What the screenshot proves: The SSH key files are present for kOps node access. Sensitive key material is redacted in the generated documentation.

```

[root@ip-172-31-10-235 .ssh]# ls
authorized_keys  id_rsa  id_rsa.pub
[root@ip-172-31-10-235 .ssh]# cat id_rsa.pub
no-port-forwarding,no-agent-forwarding,no-X11-forwarding,command="echo 'Please login as the user \"ec2-user\" rather than the user \"root\".';echo;sleep 10;exit 142" ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDFTnhKfMTDkr0gOU+PtCLL47EdaA1Rm9qjSYa9OezTXC+UgeEamK0W5MbZ1GrG+YkMQq18CZj5N1vYxANz8yzAdwwIxG0FtnqT3umghukh3BPMZmlnv0l0cCd13p+YnGK9277hdPxS+SWychu3ZU8qy2NusvNLk856tA9WpUn31JNQFNk814BeRiCtPtFNTjGXWUZDQH2WhGC8FAa47BOHgBhZZSxrgvFuoleDrqeETf1v8g1Z4Hqg31EiGEX1nkYnUm0u6M+nuSw1x1N7NSj96sQnNm7x3zRxx/0k5a5oqEh42avkh0ppq3mSjBQ4f8DfyWnAD8AefWcZKkgf/ linux
[REDACTED]
mDGgUYEGYKA+LHwcsKJ+x++fOye1EDSPb48UJZmvt6k3V6spnb1FudN19F1msXp
x4f1lKa0ta51D2diYN0HtUVRZPKDpNFaVxDseLD+MSZtg4Cd4PNagaxILYd9NBom
KBwPjFenL6gP1lZTLb8YwIKTqLnfbwIjCksFHykJu8VRTDVOI1269ZECgYEASd2f
tBHuH3jdYhZ106g8HAzGkrZgxnsCPAFVfhnHaL3KbVbketjxPluDo1+nbYnX9x+J

i-0b5fa0499df08abba (1kops)
PublicIPs: 13.126.206.155  PrivateIPs: 172.31.10.235
  
```

Figure 9. Prepare SSH keys for kOps.

Step 9 : Create the kOps cluster configuration

Implementation Action / What I did: Set the kOps cluster name to level9.k8s.local and the state store to the 1kops S3 bucket, then create the cluster configuration with kops create cluster.

What the screenshot proves: kOps creates the cluster configuration and reports the level9.k8s.local cluster.

```

WarmPool/control-plane-ap-south-1b.masters.level9.k8s.local
  Enabled      false
  MinSize      0
  AutoscalingGroup  name:control-plane-ap-south-1b.masters.level9.k8s.local id:control-plane-ap-south-1b.masters.level9.k8s.local

WarmPool/nodes-ap-south-1b.level9.k8s.local
  Enabled      false
  MinSize      0
  AutoscalingGroup  name:nodes-ap-south-1b.level9.k8s.local id:nodes-ap-south-1b.level9.k8s.local

Must specify --yes to apply changes
Cluster configuration has been created.

Suggestions:
* list clusters with: kops get cluster
* edit this cluster with: kops edit cluster level9.k8s.local
* edit your node instance group: kops edit ig --name=level9.k8s.local nodes-ap-south-1b
* edit your control-plane instance group: kops edit ig --name=level9.k8s.local control-plane-ap-south-1b

Finally configure your cluster with: kops update cluster --name level9.k8s.local --yes --admin

[root@ip-172-31-10-235 ~]#

i-0b5fa0499df08abba (1kops)
PublicIPs: 13.126.206.155  PrivateIPs: 172.31.10.235
  
```

Figure 10. Create the kOps cluster configuration.

Step 10 : Apply the kOps cluster configuration

Implementation Action / What I did: Run `kops update cluster level9.k8s.local --yes` and wait while kOps creates the AWS/Kubernetes resources. The evidence shows kOps progressing through cluster creation and exporting kubeconfig.

What the screenshot proves: kOps applies the cluster configuration and sets the `kubectl` context to `level9.k8s.local`.

```

10904 12:19:47.151652 4152 executor.go:113] Tasks: 106 done / 119 total; 2 can run
10904 12:19:47.198074 4152 executor.go:113] Tasks: 108 done / 119 total; 10 can run
10904 12:19:53.924001 4152 executor.go:113] Tasks: 118 done / 119 total; 1 can run
10904 12:19:53.955958 4152 executor.go:113] Tasks: 119 done / 119 total; 0 can run
10904 12:19:53.956110 4152 update_cluster.go:419] Exporting kubeconfig for cluster
kOps has set your kubectl context to level9.k8s.local
W0904 12:19:54.043484 4152 update_cluster.go:439] Exported kubeconfig with no user authentication; use --admin, --user or --auth-plugin flags with 'kops export kubeconfig'

Cluster is starting. It should be ready in a few minutes.

Suggestions:
* validate cluster: kops validate cluster --wait 10m
* list nodes: kubectl get nodes --show-labels
* ssh to a control-plane node: ssh -i ~/.ssh/id_rsa ubuntu@
* the ubuntu user is specific to Ubuntu. If not using Ubuntu please use the appropriate user based on your OS.
* read about installing addons at: https://kops.sigs.k8s.io/addons.

[root@ip-172-31-10-235 ~]# kops validate cluster --wait 10m
Using cluster from kubectl context: level9.k8s.local

Validating cluster level9.k8s.local

```

i-0b5fa0499df08abba (1kops)
PublicIPs: 13.126.206.155 PrivateIPs: 172.31.10.235

Figure 11. Apply the kOps cluster configuration.

Step 11 : Inspect Kubernetes system workloads

Implementation Action / What I did: After the cluster begins starting, run `kubectl get pods -A` and inspect kube-system workloads such as Cilium, CoreDNS, etcd, kube-apiserver, controller manager, scheduler and AWS EBS CSI components.

What the screenshot proves: The kube-system namespace contains the expected Kubernetes and Cilium system workloads. The evidence also shows a transient API timeout message before the pod listing.

```

E0904 13:38:10.977557 6749 memcache.go:381] "Couldn't get current server API group list" err="Get \"https://api-level9-k8s-local-i7r0me-a-fb9becbbe258a6b3.elb.ap-south-1.amazonaws.com/api?timeout=32s\": dial tcp 13.200.209.106:443: i/o timeout"
^C
[root@ip-172-31-10-235 ~]# kubectl get pods -A

```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	aws-cloud-controller-manager-sz754	1/1	Running	0	4m41s
kube-system	aws-node-termination-handler-6ff45c9b6d-2mm2g	1/1	Running	0	4m40s
kube-system	cilium-fhqv2	1/1	Running	0	3m47s
kube-system	cilium-operator-58494bd786-pdj22	1/1	Running	0	4m40s
kube-system	cilium-p76s2	1/1	Running	0	4m41s
kube-system	coredns-69f98dccfc-7cpkk	1/1	Running	0	3m18s
kube-system	coredns-69f98dccfc-tm48k	1/1	Running	0	4m40s
kube-system	coredns-autoscaler-6b68775d-zrcpj	1/1	Running	0	4m40s
kube-system	ebs-csi-controller-7ccc7fb8bb-c7v18	5/5	Running	0	4m40s
kube-system	ebs-csi-node-bzn4j	3/3	Running	0	3m47s
kube-system	ebs-csi-node-p4kh8	3/3	Running	0	4m21s
kube-system	etcd-manager-events-i-00a8e9fca71626b71	1/1	Running	0	4m16s
kube-system	etcd-manager-main-i-00a8e9fca71626b71	1/1	Running	0	4m6s
kube-system	kops-channels-i-00a8e9fca71626b71	1/1	Running	0	4m55s
kube-system	kops-controller-f9t2t	1/1	Running	0	4m41s
kube-system	kube-apiserver-i-00a8e9fca71626b71	2/2	Running	0	4m51s
kube-system	kube-controller-manager-i-00a8e9fca71626b71	1/1	Running	3 (5m8s ago)	4m46s
kube-system	kube-scheduler-i-00a8e9fca71626b71	1/1	Running	3 (5m10s ago)	4m44s

i-0b5fa0499df08abba (1kops)
PublicIPs: 13.126.206.155 PrivateIPs: 172.31.10.235

Figure 12. Inspect Kubernetes system workloads.

Step 12 : Verify EC2 nodes created by kOps

Implementation Action / What I did: Open the EC2 Instances page and confirm the kOps-created control-plane and worker infrastructure is running and passing status checks.

What the screenshot proves: The EC2 console shows the kOps infrastructure instances in the running state.

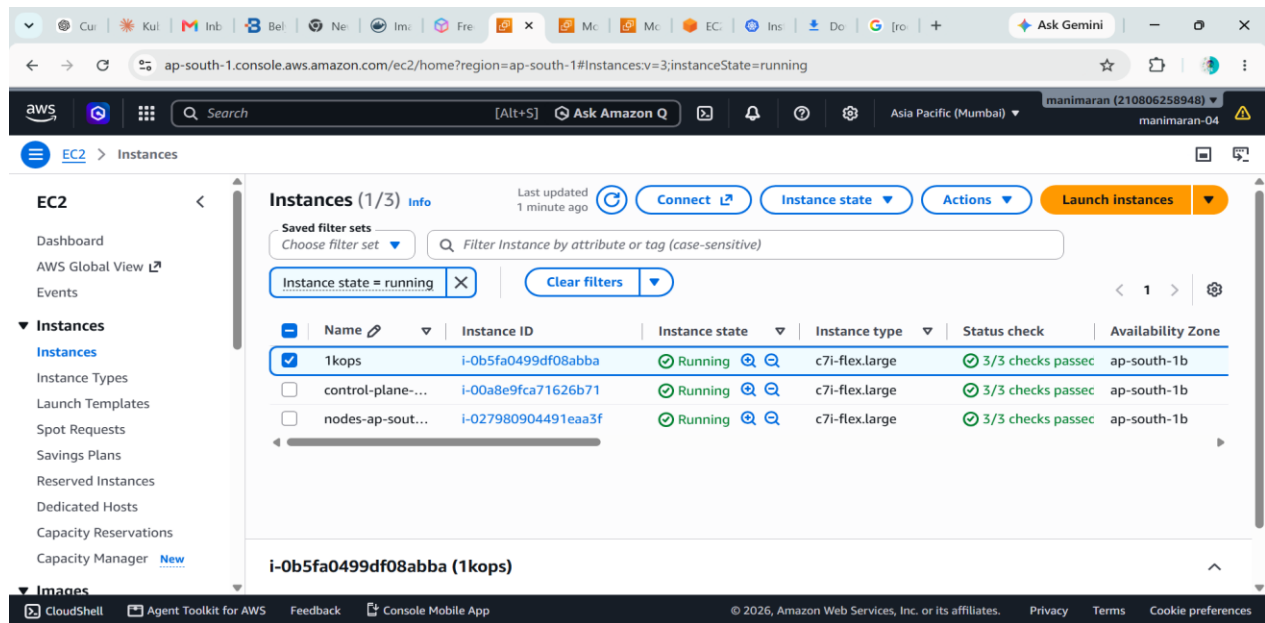


Figure 13. Verify EC2 nodes created by kOps.

Step 13 : Validate Kubernetes nodes and system components

Implementation Action / What I did: Run `kubectl get pods -A` and `kubectl get nodes`. Confirm the control-plane and worker node are Ready and inspect the running system components.

What the screenshot proves: The control-plane and worker nodes are Ready; the evidence shows Kubernetes v1.36.4 nodes and running system workloads.

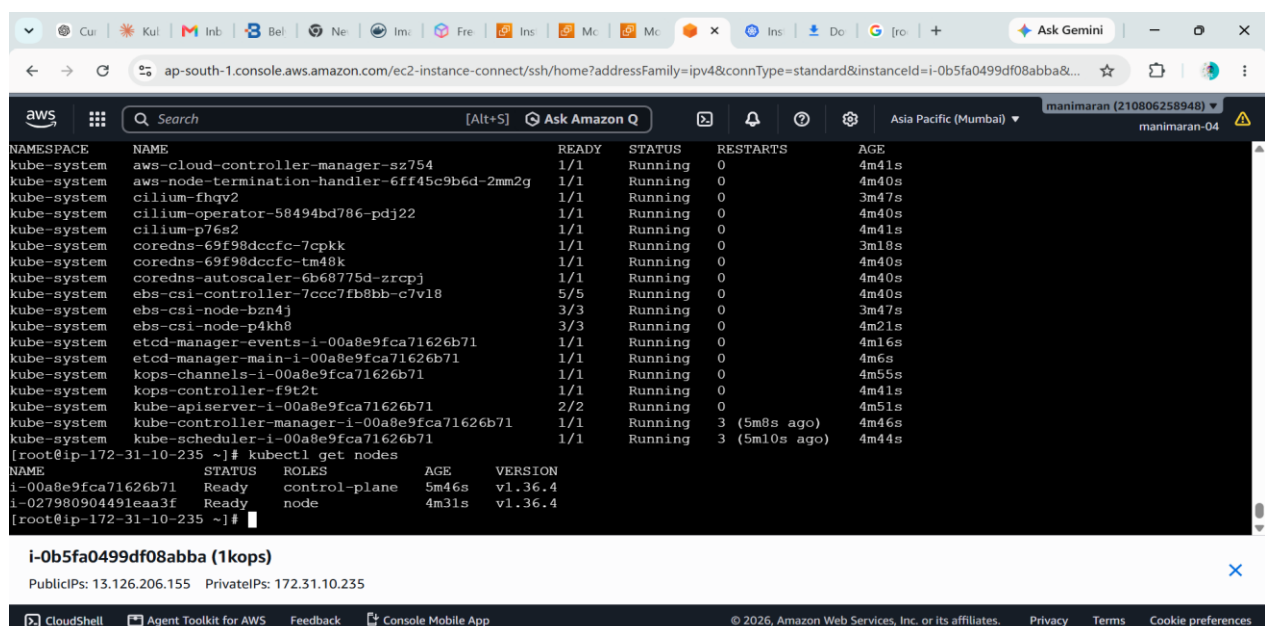


Figure 14. Kubernetes nodes and system components.

Step 14 : Test a simple Kubernetes Pod

Implementation Action / What I did: Create a temporary nginx-pod.yaml, apply it with `kubect apply -f nginx-pod.yaml`, verify the pod reaches Running, and then delete the test pod.

What the screenshot proves: The nginx test pod is created, reaches 1/1 Running, and is successfully removed.

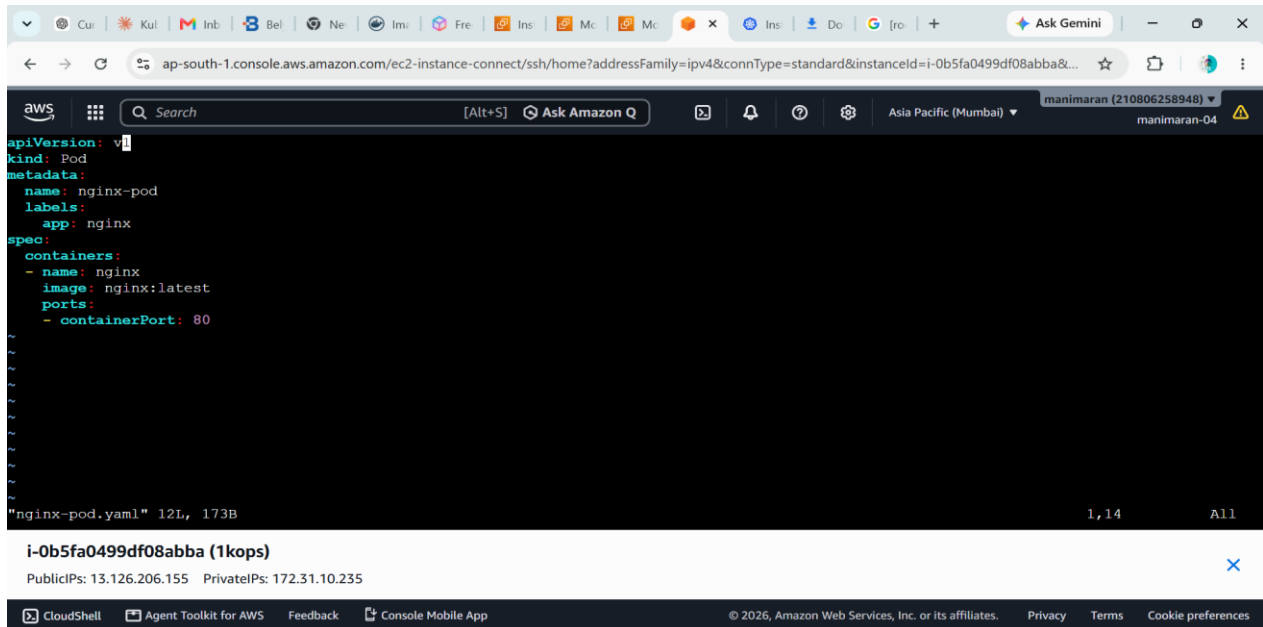


Figure 15. Test a simple Kubernetes Pod.

Step 15 : Check cluster connectivity and repeat the basic pod test

Implementation Action / What I did: Use `kubectl cluster-info` to verify the control plane/CoreDNS endpoints, then apply the temporary nginx manifest and inspect the pod lifecycle.

What the screenshot proves: `kubectl` reports the Kubernetes control plane/CoreDNS endpoints and the nginx test pod runs successfully before deletion.

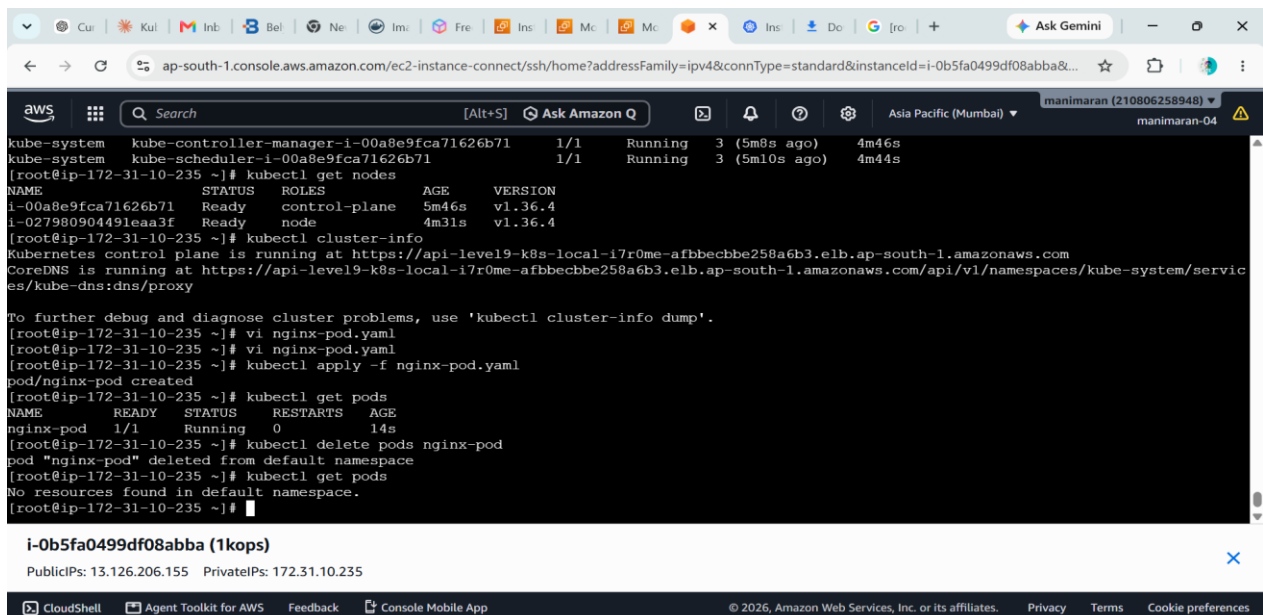


Figure 16. Check cluster connectivity and repeat the basic pod test.

Step 16 : Create the Docker Hub imagePullSecret

Implementation Action / What I did: Create the Kubernetes docker-registry secret named dockerhub-secret for Docker Hub authentication. The generated documentation redacts credential values visible in the original screenshot.

What the screenshot proves: The dockerhub-secret is created for private-registry authentication.

```

I0907 11:18:32.637759 3929 executor.go:113] Tasks: 102 done / 119 total; 4 can run
I0907 11:18:32.939851 3929 executor.go:113] Tasks: 106 done / 119 total; 2 can run
I0907 11:18:33.009413 3929 executor.go:113] Tasks: 108 done / 119 total; 10 can run
I0907 11:18:39.481406 3929 executor.go:113] Tasks: 118 done / 119 total; 1 can run
I0907 11:18:39.513922 3929 executor.go:113] Tasks: 119 done / 119 total; 0 can run
I0907 11:18:39.514054 3929 update_cluster.go:419] Exporting kubeconfig for cluster
kOps has set your kubectl context to level9.k8s.local

Cluster is starting. It should be ready in a few minutes.

Suggestions:
* validate cluster: kops validate cluster --wait 10m
* list nodes: kubectl get nodes --show-labels
* ssh to a control-plane node: ssh -i ~/.ssh/id_rsa ubuntu@
REDACTEDCREDENTIAL

--docker-password=dckr_pat_u1ajT0ZkPY5KjnigpU1DUor28AA \
--docker-email=maran3154@gmail.com
secret/dockerhub-secret created
[root@ip-172-31-13-119 ~]#

```

i-071a7493795cbd50d (kops)
PublicIPs: 52.66.14.114 PrivateIPs: 172.31.13.119

Figure 17. Create the Docker Hub imagePullSecret.

Step 17 : Verify the Docker Hub imagePullSecret

Implementation Action / What I did: Run kubectl get secrets and confirm dockerhub-secret exists with type kubernetes.io/dockerconfigjson.

What the screenshot proves: The Docker Hub registry secret is present and has the expected Kubernetes secret type.

```

I0907 11:18:39.481406 3929 executor.go:113] Tasks: 118 done / 119 total; 1 can run
I0907 11:18:39.513922 3929 executor.go:113] Tasks: 119 done / 119 total; 0 can run
I0907 11:18:39.514054 3929 update_cluster.go:419] Exporting kubeconfig for cluster
kOps has set your kubectl context to level9.k8s.local

Cluster is starting. It should be ready in a few minutes.

Suggestions:
* validate cluster: kops validate cluster --wait 10m
* list nodes: kubectl get nodes --show-labels
* ssh to a control-plane node: ssh -i ~/.ssh/id_rsa ubuntu@
* the ubuntu user is specific to Ubuntu. If not using Ubuntu please use the appropriate user based on your OS.
* read about installing addons at: https://kops.sigs.k8s.io/addons.
REDACTEDCREDENTIAL

[root@ip-172-31-13-119 ~]# kubectl get secrets
NAME                TYPE                                DATA  AGE
dockerhub-secret    kubernetes.io/dockerconfigjson    1      19s
[root@ip-172-31-13-119 ~]#

```

i-071a7493795cbd50d (kops)
PublicIPs: 52.66.14.114 PrivateIPs: 172.31.13.119

Figure 18. Verify the Docker Hub imagePullSecret.

Step 18 : Prepare the calculator Deployment manifest

Implementation Action / What I did: Create calculator-deployment.yaml defining the calculator-app Deployment, three replicas, the Docker Hub image and the dockerhub-secret imagePullSecret. The application container listens on port 5000.

What the screenshot proves: The Deployment manifest contains calculator-app, three replicas, the registry secret, image reference and port 5000.

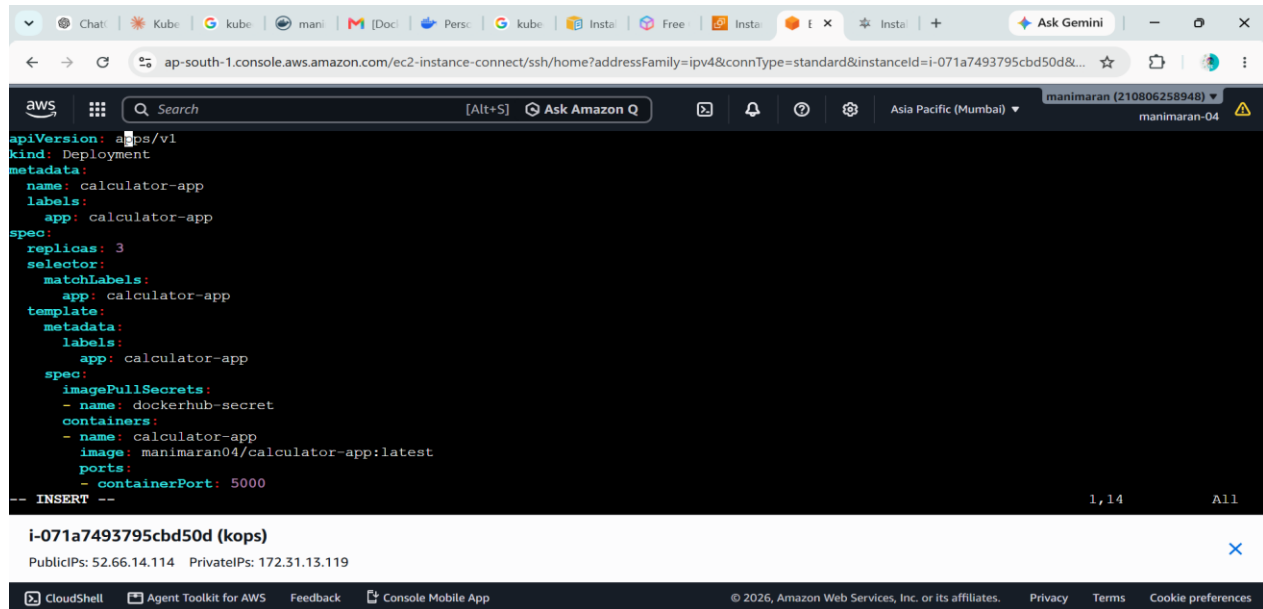


Figure 19. Prepare the calculator Deployment manifest.

Step 19 : Create the calculator application Deployment

Implementation Action / What I did: Run `kubectl apply -f calculator-deployment.yaml` and `kubectl get pods -w`. Wait until all three calculator-app pods become Running and ready.

What the screenshot proves: Three calculator application pods reach 1/1 Running.

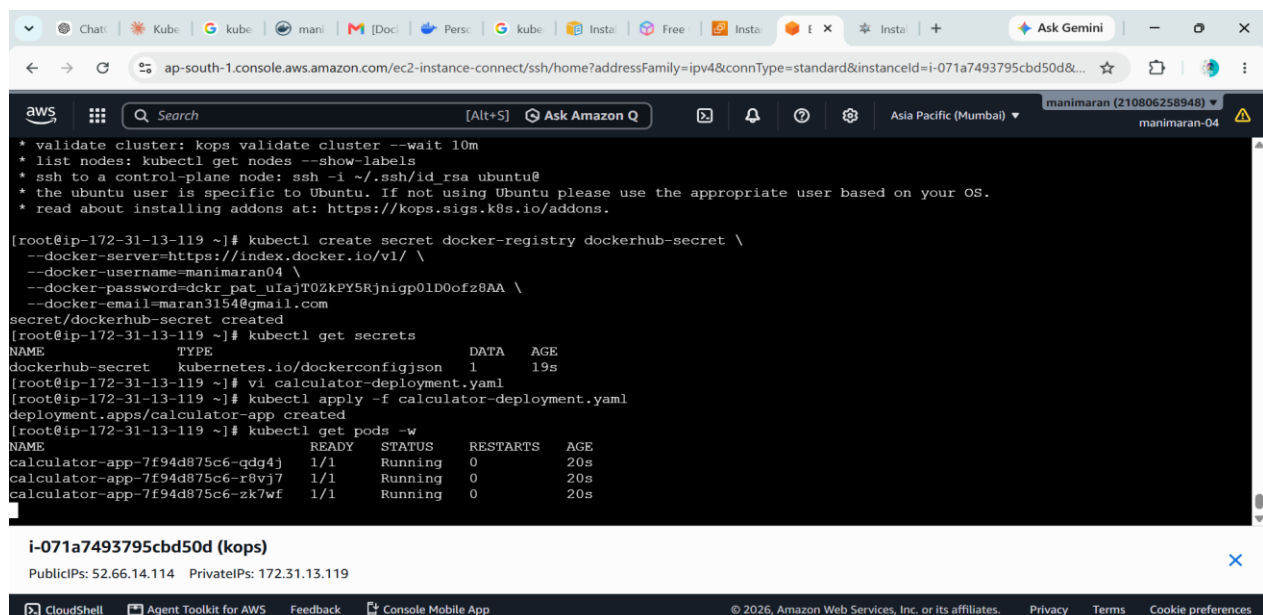


Figure 20. Create the calculator application Deployment.

Step 20 : Verify the Deployment ReplicaSet

Implementation Action / What I did: Run `kubectl get rs -w` and confirm the calculator-app ReplicaSet reports desired, current and ready counts of 3.

What the screenshot proves: The calculator-app ReplicaSet shows 3 desired, 3 current and 3 ready replicas.

```

* the ubuntu user is specific to Ubuntu. If not using Ubuntu please use the appropriate user based on your OS.
* read about installing addons at: https://kops.sigs.k8s.io/addons.

[root@ip-172-31-13-119 ~]# kubectl create secret docker-registry dockerhub-secret \
--docker-server=https://index.docker.io/v1/ \
--docker-username=manimaran04 \
--docker-password=dckr_pat_uIajT0ZkPY5Rjnigp0lD0ofz8AA \
--docker-email=maran3154@gmail.com
secret/dockerhub-secret created
[root@ip-172-31-13-119 ~]# kubectl get secrets
NAME                                TYPE                                DATA  AGE
dockerhub-secret                    kubernetes.io/dockerconfigjson     1      19s
[root@ip-172-31-13-119 ~]# vi calculator-deployment.yaml
[root@ip-172-31-13-119 ~]# kubectl apply -f calculator-deployment.yaml
deployment.apps/calculator-app created
[root@ip-172-31-13-119 ~]# kubectl get pods -w
NAME                                READY  STATUS   RESTARTS  AGE
calculator-app-7f94d875c6-qdg4j     1/1    Running  0          20s
calculator-app-7f94d875c6-r8vj7     1/1    Running  0          20s
calculator-app-7f94d875c6-zk7wf     1/1    Running  0          20s
^C[root@ip-172-31-13-119 ~]# kubectl get rs -w
NAME                                DESIRED  CURRENT  READY  AGE
calculator-app-7f94d875c6           3         3         3      42s

```

Figure 21. Verify the Deployment ReplicaSet.

Step 21 : Prepare the NodePort Service manifest

Implementation Action / What I did: Create calculator-service.yaml with Service type NodePort, selector app=calculator-app, service port 80, target/container port 5000 and NodePort 30080.

What the screenshot proves: The Service manifest defines calculator-service and the 30080 → 5000 exposure path.

```

apiVersion: v1
kind: Service
metadata:
  name: calculator-service
spec:
  type: NodePort
  selector:
    app: calculator-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      nodePort: 30080

```

Figure 22. Prepare the NodePort Service manifest.

Step 22 : Create and verify the NodePort Service

Implementation Action / What I did: Run `kubectl apply -f calculator-service.yaml` and `kubectl get svc calculator-service`. Confirm the service is NodePort and the assigned port is 30080.

What the screenshot proves: calculator-service is created as a NodePort service and exposes port 30080 to the application port 5000.

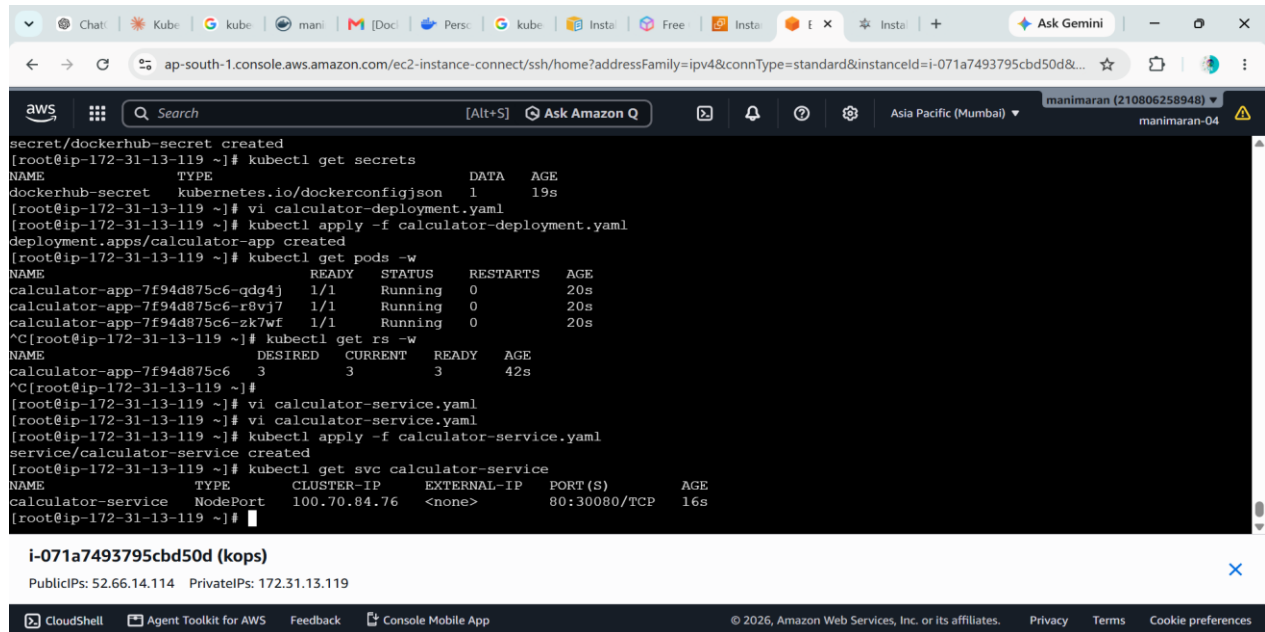


Figure 23. Create and verify the NodePort Service.

Step 23 : Allow NodePort traffic in the worker security group

Implementation Action / What I did: Open the EC2 security group inbound rules and add the required TCP 30080 rule so external browser traffic can reach the NodePort on the worker node.

What the screenshot proves: The security group contains the TCP 30080 inbound rule shown in the evidence.

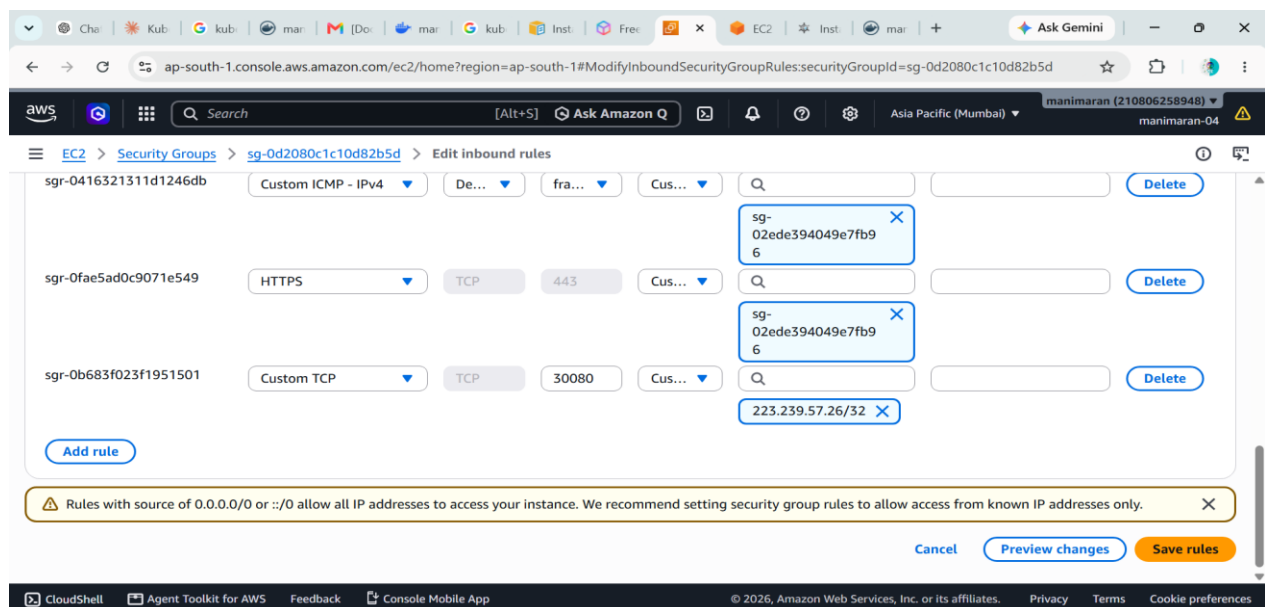


Figure 24. Allow NodePort traffic in the worker security group.

Step 24 : Identify the control-plane/cluster instance address

Implementation Action / What I did: Open the EC2 instance details and record the public IP needed for cluster administration/validation. Do not expose unnecessary public addresses in public documentation.

What the screenshot proves: The EC2 console shows the running kOps cluster instance and its network details.

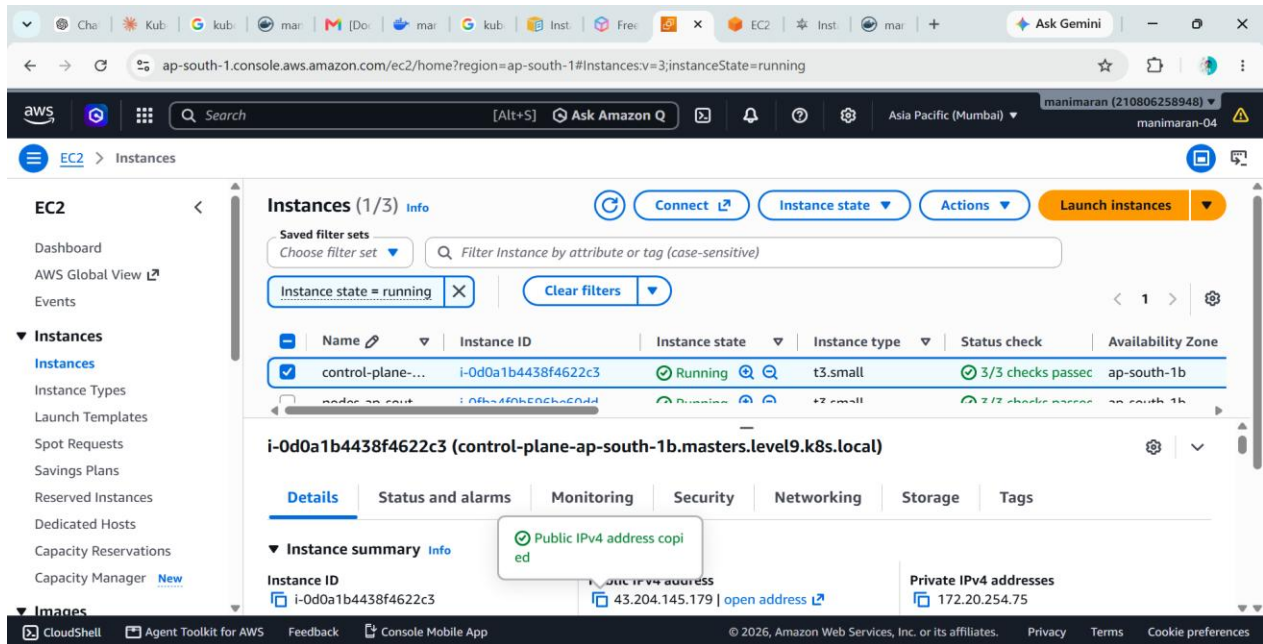


Figure 25. Identify the control-plane/cluster instance address.

Step 25 : Test the calculator through NodePort

Implementation Action / What I did: Open `http://<worker-node-public-ip>:30080` in a browser and enter 2.0 and 3.0, then select Calculate.

What the screenshot proves: The Flask calculator loads and returns Result: 5.0.

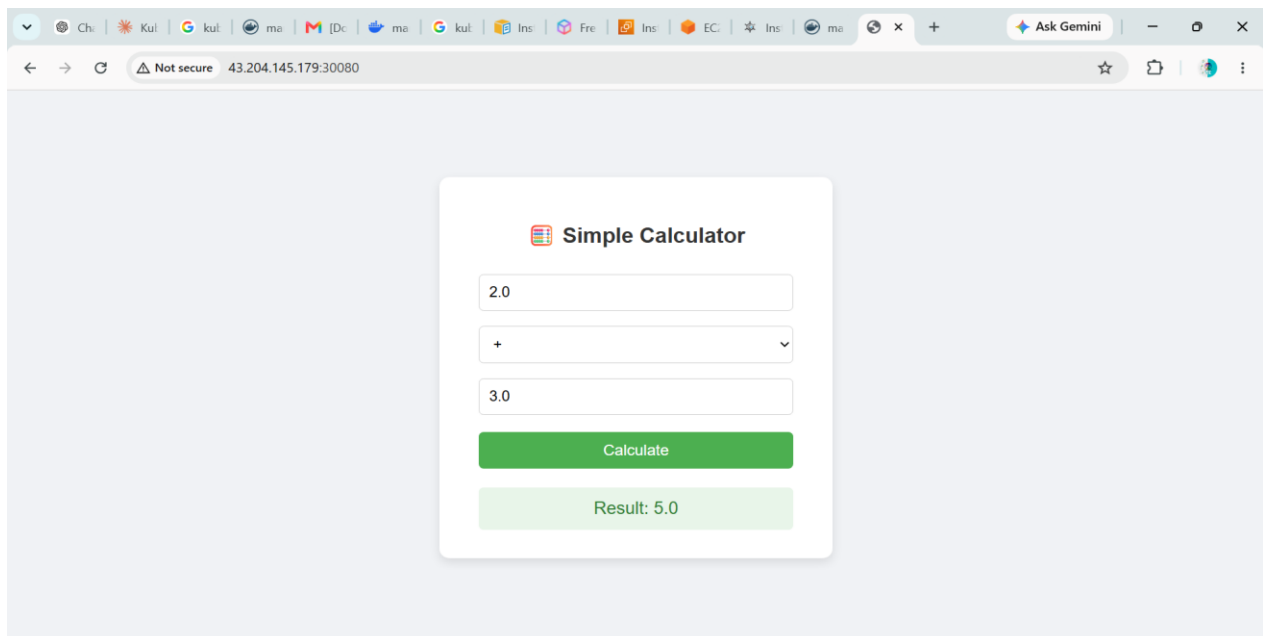


Figure 26. Test the calculator through NodePort.

Step 26 : Update the calculator image/version

Implementation Action / What I did: Edit calculator-deployment.yaml to use the updated calculator application image/version (the supplied evidence shows manimaran04/calculator-app:v2) and apply the updated Deployment.

What the screenshot proves: Kubernetes accepts the updated Deployment configuration and starts the new application version.

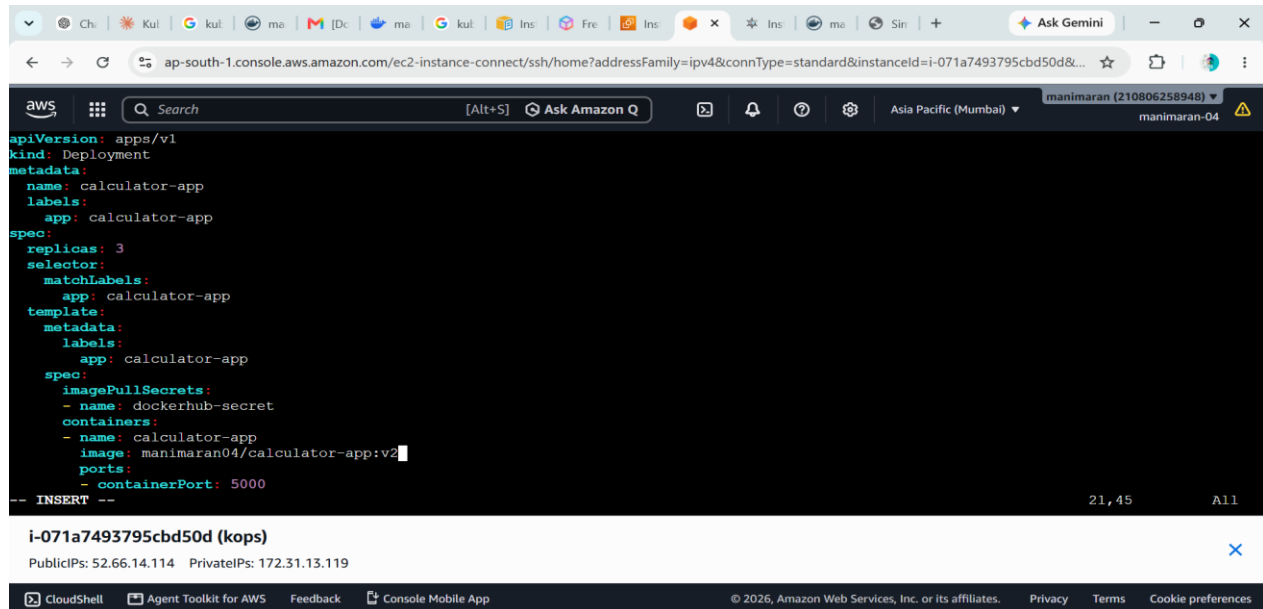


Figure 27. Update the calculator image/version.

Step 27 : Observe rollout status and ReplicaSets

Implementation Action / What I did: Run `kubectl get pods -o wide`, `kubectl rollout status deployment/calculator-app`, `kubectl get deployments` and `kubectl get replicaset`s. Confirm the new ReplicaSet becomes ready and the rollout completes.

What the screenshot proves: The deployment reports 3/3 ready and the ReplicaSet listing shows the active version with 3 ready replicas.

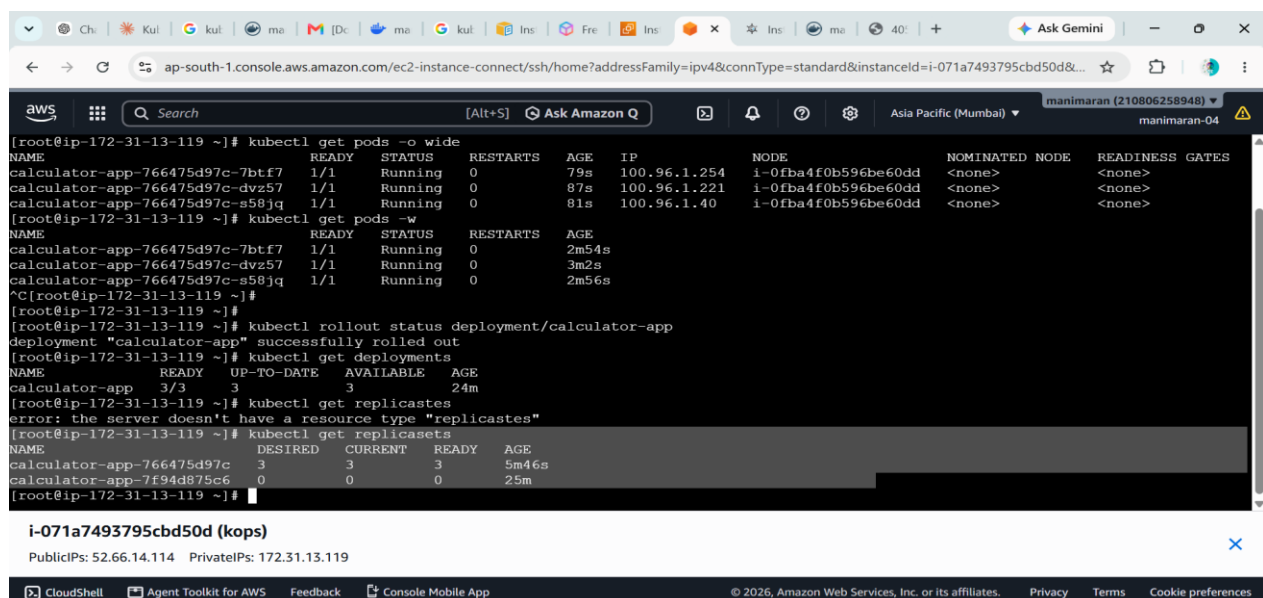


Figure 28. Observe rollout status and ReplicaSets.

Step 28 : Observe the old ReplicaSet being scaled down

Implementation Action / What I did: Watch `kubectl get rs -w` and `kubectl get pods -w` during the rolling update. Verify the old ReplicaSet reaches 0/0/0 while the new ReplicaSet maintains 3/3/3.

What the screenshot proves: The evidence shows the old ReplicaSet at 0 desired/current/ready and the active ReplicaSet at 3/3/3.

```

calculator-app-7f94d875c6-bjppc 1/1 Running 0 11s
calculator-app-7f94d875c6-dfwhx 1/1 Running 0 13s
calculator-app-7f94d875c6-pqjmq 1/1 Running 0 10s
calculator-app-766475d97c-7bt17 0/1 Error 0 8m57s
calculator-app-766475d97c-7bt17 0/1 Error 0 8m57s
calculator-app-766475d97c-7bt17 0/1 Error 0 8m57s
calculator-app-766475d97c-dvz57 0/1 Error 0 9m7s
calculator-app-766475d97c-dvz57 0/1 Error 0 9m7s
calculator-app-766475d97c-dvz57 0/1 Error 0 9m7s
calculator-app-766475d97c-s58jq 0/1 Error 0 9m3s
calculator-app-766475d97c-s58jq 0/1 Error 0 9m3s
calculator-app-766475d97c-s58jq 0/1 Error 0 9m3s
calculator-app-7f94d875c6-bjppc 1/1 Running 0 72s
calculator-app-7f94d875c6-pqjmq 1/1 Running 0 71s
calculator-app-7f94d875c6-dfwhx 1/1 Running 0 81s
^C[root@ip-172-31-13-119 ~]# kubectl get rs -w
NAME                                DESIRED    CURRENT    READY    AGE
calculator-app-766475d97c            0           0           0        10m
calculator-app-7f94d875c6            3           3           3        30m
^C[root@ip-172-31-13-119 ~]#
[root@ip-172-31-13-119 ~]# vi calculator-deployment.yaml
[root@ip-172-31-13-119 ~]# kubectl apply -f calculator-deployment.yaml
deployment.apps/calculator-app configured
[root@ip-172-31-13-119 ~]#

```

Figure 29. Observe the old ReplicaSet being scaled down.

Step 29 : Complete the version change and verify the new ReplicaSet

Implementation Action / What I did: Apply the calculator Deployment configuration and observe the ReplicaSet transition until the selected version has three ready pods.

What the screenshot proves: The new ReplicaSet reaches 3/3/3 and the corresponding pods are Running.

```

calculator-app-7f94d875c6-pqjmq 1/1 Running 0 71s
calculator-app-7f94d875c6-dfwhx 1/1 Running 0 81s
^C[root@ip-172-31-13-119 ~]# kubectl get rs -w
NAME                                DESIRED    CURRENT    READY    AGE
calculator-app-766475d97c            0           0           0        10m
calculator-app-7f94d875c6            3           3           3        30m
^C[root@ip-172-31-13-119 ~]#
[root@ip-172-31-13-119 ~]# vi calculator-deployment.yaml
[root@ip-172-31-13-119 ~]# kubectl apply -f calculator-deployment.yaml
deployment.apps/calculator-app configured
[root@ip-172-31-13-119 ~]# kubectl get rs -w
NAME                                DESIRED    CURRENT    READY    AGE
calculator-app-766475d97c            0           0           0        10m
calculator-app-7f94d875c6            3           3           3        30m
^C[root@ip-172-31-13-119 ~]#
[root@ip-172-31-13-119 ~]# kubectl get pods -w
NAME                                READY     STATUS    RESTARTS   AGE
calculator-app-766475d97c-g9wqw 1/1 Running 0          54s
calculator-app-766475d97c-gbv45 1/1 Running 0          56s
calculator-app-766475d97c-nx77s 1/1 Running 0          58s

```

Figure 30. Complete the version change and verify the new ReplicaSet.

Step 30 : Validate the calculator after the update

Implementation Action / What I did: Open the NodePort application again and test a calculation such as 13×9 .

What the screenshot proves: The browser calculator returns 117, demonstrating application functionality after the update.

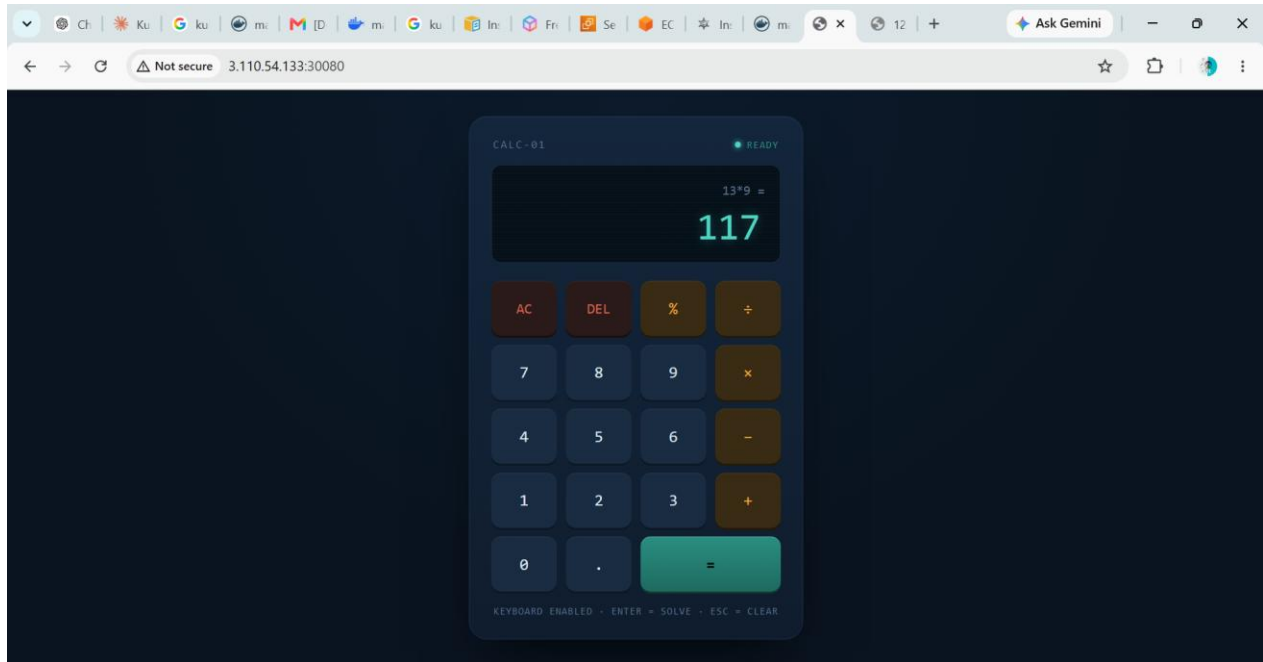


Figure 31. Validate the calculator after the update.

Step 31 : Rollback the Deployment

Implementation Action / What I did: Run `kubectl rollout undo deployment/calculator-app` to restore the previous Deployment revision.

What the screenshot proves: Kubernetes reports that calculator-app has been rolled back.

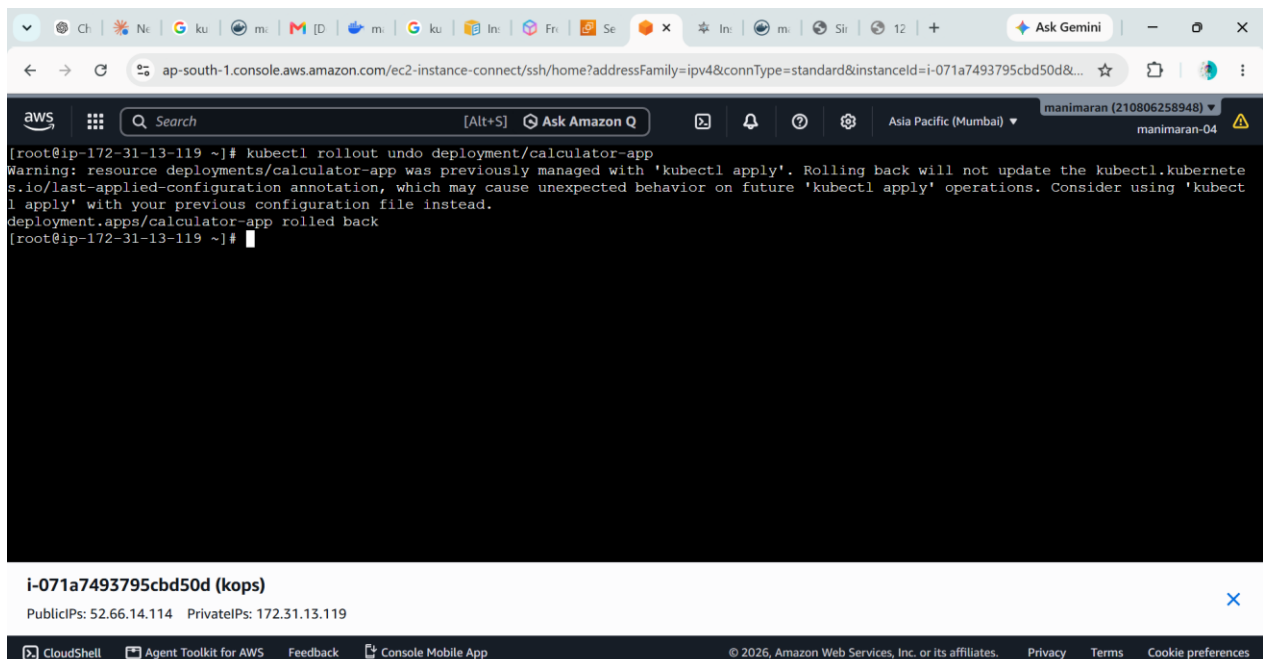


Figure 32. Rollback the Deployment.

Step 32 : Verify ReplicaSets after rollback

Implementation Action / What I did: Run `kubectl get rs -o wide` and inspect the desired/current/ready counts. Confirm the restored ReplicaSet is active with three ready replicas.

What the screenshot proves: The ReplicaSet evidence shows the restored application version with 3 ready replicas while the other revision is scaled down.

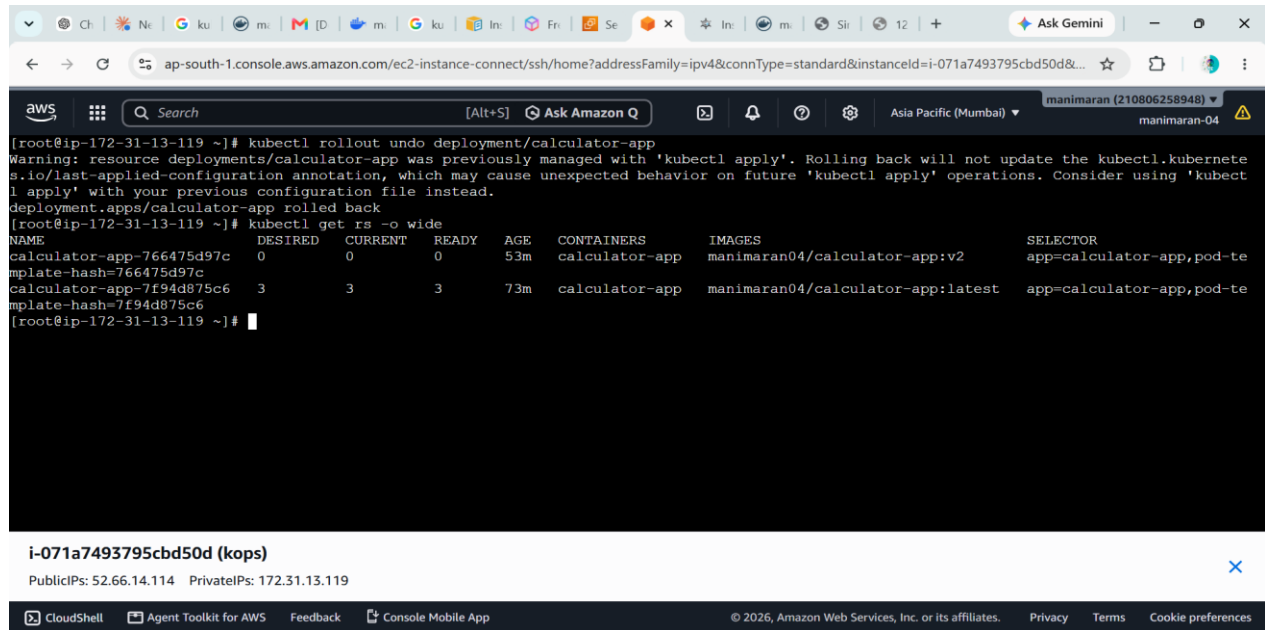


Figure 33. Verify ReplicaSets after rollback.

Step 33 : Verify the calculator after rollback

Implementation Action / What I did: Open the NodePort URL again and perform another calculation, such as $9.0 + 2004.0$.

What the screenshot proves: The browser calculator works after rollback and returns 2013.0.

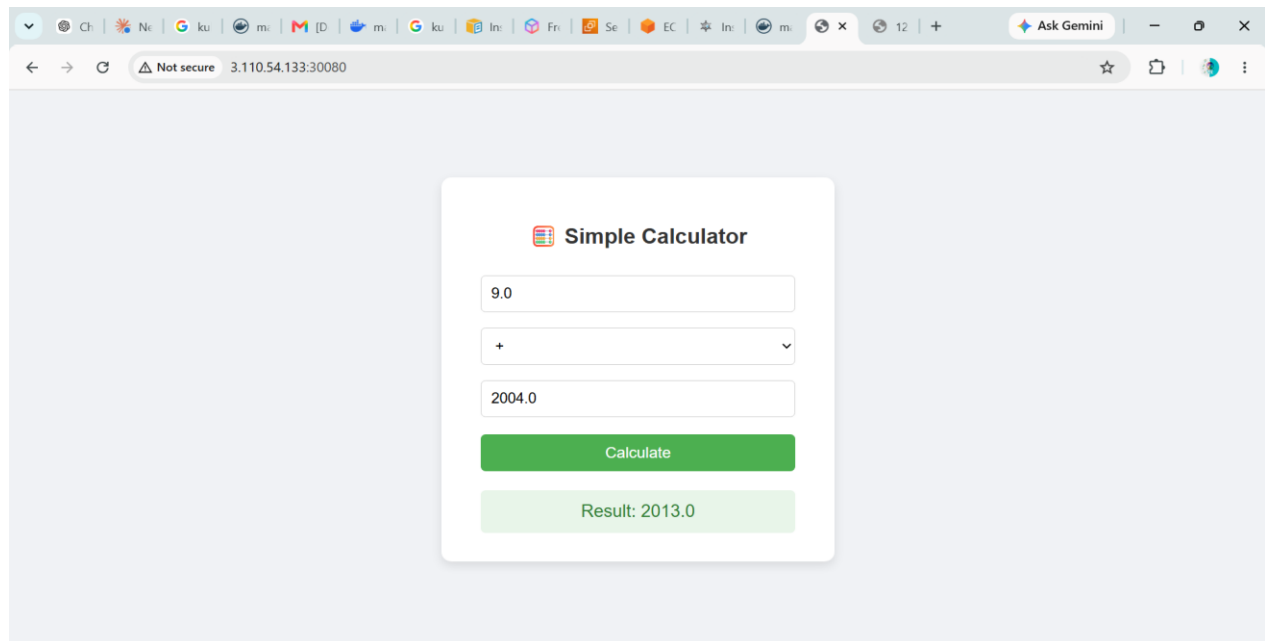


Figure 34. Verify the calculator after rollback.

Step 34 : Clean up the kOps cluster

Implementation Action / What I did: After completing testing, remove the Kubernetes cluster with the appropriate kOps deletion command and wait for AWS resources to be removed. The evidence shows kOps retrying dependent resource deletion before completing the cleanup.

What the screenshot proves: kOps deletes the cluster, removes the kubectl configuration for level9.k8s.local and reports the cluster deleted.

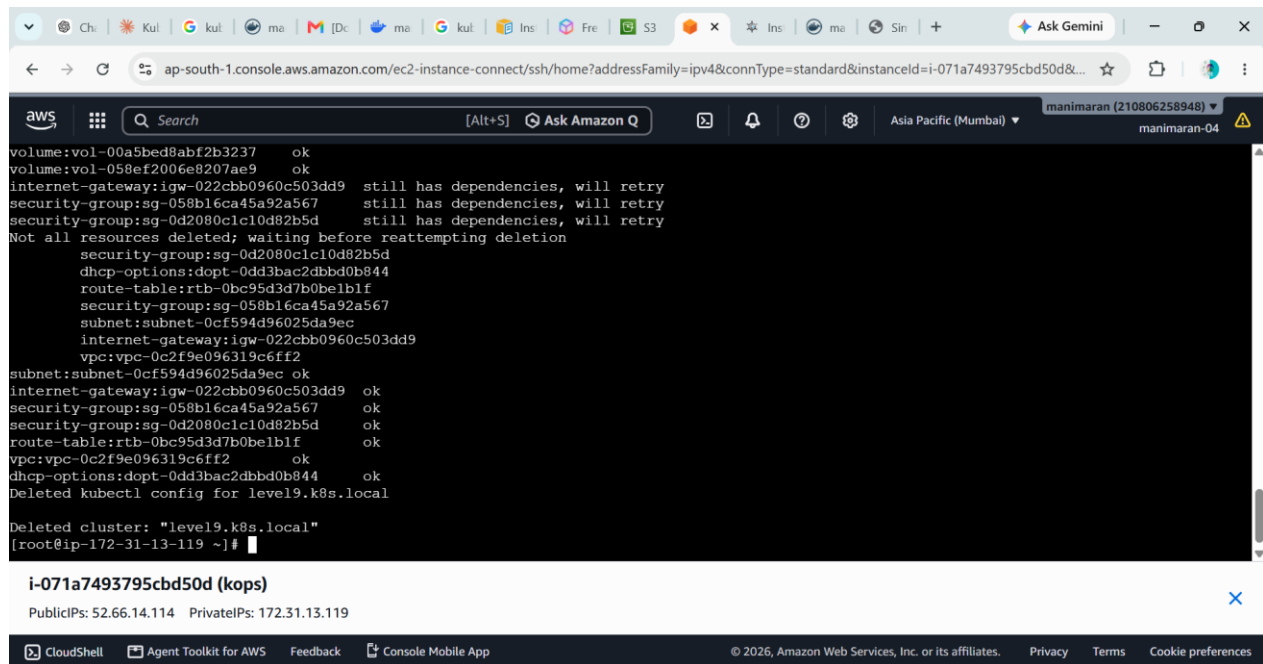


Figure 35. Clean up the kOps cluster.

10. Kubernetes Deployment and Service Design

The calculator application is represented by a Deployment named calculator-app with three replicas. Kubernetes creates and manages ReplicaSets for application revisions. The image is pulled from Docker Hub using dockerhub-secret. The Service calculator-service is a NodePort service; the supplied evidence shows port mapping 80:30080/TCP and the application container port is 5000.

- Deployment → 3 Pods → Flask container :5000
- NodePort 30080 → Service → Pods
- imagePullSecret → Docker Hub authentication

11. Rolling Update and Rollback

The project demonstrates Kubernetes rolling update behavior by changing the calculator application image/version. The supplied evidence shows a new ReplicaSet becoming ready while the previous ReplicaSet is reduced to zero. The observed final active state in the source report is calculator-app-766475d97c: 0/0/0 and calculator-app-7f94d875c6: 3/3/3. A subsequent kubectl rollout undo deployment/calculator-app restores the previous revision, after which ReplicaSet and browser validation confirm the application is operational.

12. Application Request Flow

Browser → Worker Node Public IP:30080 → calculator-service → calculator-app Deployment → 3 Pods → Flask container :5000

13. Validation and Verification Checklist

- ✓ kOps installed and version verified
- ✓ kubectl available and version verified
- ✓ S3 state store configured
- ✓ level9.k8s.local cluster created/validated
- ✓ Control-plane and worker infrastructure operating
- ✓ Cilium networking components running
- ✓ calculator-app Deployment created
- ✓ 3/3 application replicas ready
- ✓ calculator-service NodePort 30080 created
- ✓ Calculator accessible from browser
- ✓ Rolling update tested
- ✓ Rollback tested
- ✓ Cluster cleanup completed

14. Troubleshooting and Operational Notes

- The source evidence includes a transient Kubernetes API timeout message during early validation; the subsequent pod listing shows system workloads running.
- A temporary nginx Pod was used to verify basic Kubernetes scheduling and pod lifecycle before deploying the calculator application.
- The source evidence shows an initial kubectl resource-type typo (`replicastes`) before the correct `kubectl get replicaset` command was used.
- The NodePort security-group rule is required for browser access to TCP 30080.
- Rolling-update evidence shows old and new ReplicaSets during the transition; the final active revision maintains three ready pods.
- kOps cluster deletion may take time while dependent AWS resources are retried and removed.

15. Security Notes

The original supplied screenshots contain AWS access-key material, AWS secret-key material, SSH/private-key related content and Docker Hub registry credential material. These values are not reproduced as readable text in this generated documentation; credential-bearing screenshots are redacted. Any credentials that were exposed in the original screenshots should be revoked/rotated immediately and must not be committed to GitHub. Use IAM roles or short-lived credentials where possible.

16. Key Lessons Learned

- kOps can automate Kubernetes cluster lifecycle management on AWS.
- Amazon S3 can be used as the kOps state store.
- Cilium provides the cluster networking layer.
- Deployments maintain desired replicas through ReplicaSets.
- NodePort exposes a Kubernetes Service outside the cluster on a node port.
- imagePullSecrets provide registry authentication for private images.
- ReplicaSets provide visible evidence of rolling updates and rollback.
- kubectl inspection and cluster validation are essential for troubleshooting.

17. Interview / Viva Explanation

I deployed a Kubernetes cluster on AWS using kOps. The cluster uses Amazon S3 as its state store and runs on EC2 infrastructure. Cilium provides cluster networking. I deployed a Flask calculator application with three replicas. The image is pulled from Docker Hub using an imagePullSecret. I exposed the application using a NodePort service on port 30080, which forwards requests to the application container on port 5000. I verified the application from a browser and tested Kubernetes rolling update and rollback by observing ReplicaSets and pod readiness.

18. Conclusion

The Level 9 project demonstrates Kubernetes cluster administration and application deployment on AWS. The evidence covers kOps, EC2, S3 state storage, Cilium networking, Docker Hub authentication, Kubernetes Deployment, Pods, ReplicaSets, NodePort service exposure, browser testing, rolling updates and rollback. It provides practical experience with the complete path from AWS infrastructure to a working containerized application.

Appendix – Complete Project Flow

AWS EC2 administration → AWS CLI → S3 1kops state store → kOps level9.k8s.local → Control Plane + Worker → Cilium → Docker Hub imagePullSecret → calculator-app Deployment (3 replicas) → calculator-service NodePort 30080 → Flask :5000 → Browser validation → Rolling update → ReplicaSet transition → Rollback → Cluster cleanup